

Trekking Management Application - V2– Project Report

Author:

Name: Samprikta Malik

Roll Number: 21f3002038

Email: 21f3002038@ds.study.iitm.ac.in

Github: [samprikta1507/trekking-management-app-v2.git](https://github.com/samprikta1507/trekking-management-app-v2.git)

Description:

This is a comprehensive web application designed to manage trekking expeditions efficiently. It replaces manual tracking with a centralized, role-based platform built for three types of users: Admin, Trek Staff, and Users (Trekksers).

Admin:

- Can create, update, and remove trekking routes.
- Can add new trek staff and assign them to specific treks.
- Can view and manage all users, staff, and bookings across the platform.
- Can view summary statistics and monthly activity reports.

Trek Staff:

- Can view only the treks assigned to them by the Admin.
- Can manage their trek details, like changing the trek status (Pending / Open / Completed).
- Can view the list of registered participants for their specific treks.

Users (Trekksers):

- Can view, search, and filter approved/open treks based on difficulty, location, and duration.
- Can book available treks (the system automatically prevents overbooking beyond available slots).
- Can view their real-time booking status and complete trekking history.

Technologies:

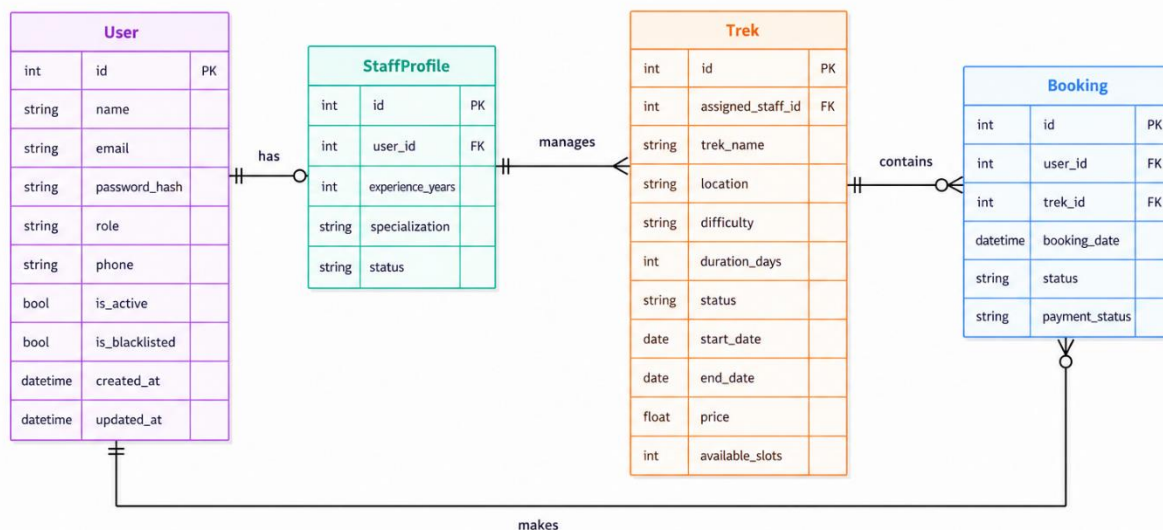
Backend:

- Flask: Web framework for Python used to build the backend RESTful APIs.
- Flask-JWT-Extended: Manages user sessions, secure authentication, and role-based access.
- SQLAlchemy: ORM (Object Relational Mapping) to interact with the relational database.
- SQLite: Embedded SQL database used for storing all application data.
- Redis: In-memory data store used for caching frequently accessed data and acting as a message broker.
- Celery: Asynchronous task queue used for handling background batch jobs like daily reminders and monthly reports.

Frontend:

- VueJS: Core JavaScript framework used for building the reactive user interface.
- Vite: Build tool and bundler used for compiling the Vue application.
- Bootstrap: CSS framework used for responsive design and layout styling.
- Axios: Promise-based HTTP client used to connect the Vue frontend to the Flask backend.

ER-Diagram:



API Design:

Authentication Operations:

- **POST /api/register** : Registers a new standard user (Trekker).
- **POST /api/login** : Authenticates a user/admin/staff and returns a JWT access token.

Admin Operations:

- **GET /api/admin/dashboard** : Verifies admin access and loads dashboard data.
- **GET /api/admin/summary** : Retrieves overall summary statistics (total treks, users, staff, bookings).
- **GET /api/admin/treks** : Retrieves a list of all trekking routes.
- **POST /api/admin/create-trek** : Creates a new trekking route.
- **PUT /api/admin/update-trek/<int:trek_id>** : Updates details of an existing trek.
- **DELETE /api/admin/delete-trek/<int:trek_id>** : Removes a specific trek from the system.
- **GET /api/admin/staff** : Retrieves a list of all registered staff profiles.
- **POST /api/admin/create-staff** : Creates a new staff member account.
- **PUT /api/admin/update-staff/<int:staff_id>** : Updates an existing staff member's details.
- **DELETE /api/admin/delete-staff/<int:staff_id>** : Deletes a staff member account.
- **GET /api/admin/users** : Retrieves a comprehensive list of all registered users.
- **PUT /api/admin/toggle-blacklist/<int:user_id>** : Toggles the active/blacklisted status of a user.
- **GET /api/admin/all-bookings** : Retrieves a list of all system-wide bookings.

Trek Staff Operations:

- **GET /api/staff/my-treks** : Fetches the specific treks assigned to the authenticated staff member.
- **GET /api/staff/participants/<int:trek_id>** : Retrieves the list of users and payment statuses for an assigned trek.
- **PUT api/staff/update-status/<int:trek_id>**: Updates the operational status of an assigned trek. (Open/Closed/Completed)

User (Trekker) Operations:

- **GET /api/user/dashboard** : Verifies user access and loads dashboard data.
- **GET /api/user/profile** : Retrieves the authenticated user's profile details.

- **PUT /api/user/update-profile** : Updates the user's profile information.
- **GET /api/user/treks** : Retrieves a list of all available treks for booking.
- **POST /api/user/book-trek/<int:trek_id>** : Submits a new booking request for a specific trek.
- **GET /api/user/my-bookings** : Fetches the complete booking history for the authenticated user.
- **PUT /api/user/pay-booking/<int:booking_id>** : Processes payment status for a specific booking.
- **PUT /api/user/cancel-booking/<int:booking_id>** : Cancels an existing booking and frees up the slot.
- **GET /api/user/trigger-export** : Triggers an asynchronous Celery task to export the user's booking history as a CSV file.

Architecture:

The application follows a client-server architecture, separating the user interface from the data processing logic.

Overall Project Architecture:

- **Design:** The Vue frontend operates independently from the Flask backend, communicating securely via RESTful APIs using Axios and JWT tokens.
- **Flask Blueprint Structure:** The backend uses modular blueprints to separate API routes by user role, ensuring clean and maintainable code.
- **Asynchronous Processing:** Redis and Celery are integrated to handle long-running background tasks without freezing the main application.

Backend Directory Structure (Flask):

- **models.py:** Contains the SQLAlchemy database models (User, StaffProfile, Trek, Booking) and their relationships.
- **routes/:** A folder containing modularized API endpoints separated by functionality (e.g., `admin_routes.py`, `user_routes.py`, `staff_routes.py`).
- **tasks/:** Contains Celery tasks for background processing, such as generating monthly activity reports and exporting user CSV files.
- **app.py:** The main entry point. It initializes the Flask app, configures the SQLite database, sets up JWT authentication, and registers all blueprints.

Frontend Directory Structure (Vue 3 + Vite):

- **src/components/:** Holds reusable UI components like `AdminDashboard.vue`, `StaffManagement.vue`, and `BookingHistory.vue`.
- **src/views/:** Contains the main page layouts (e.g., `LandingView.vue`, `AdminView.vue`) that utilize the smaller components.
- **src/router/index.js:** Manages client-side navigation, establishing paths between views and protecting restricted routes using navigation guards.

% of AI Used:

I utilized AI for approximately 25-30% of the project development. It was primarily used as a debugging assistant and learning tool to resolve Vue.js reactivity and state management issues, troubleshoot Flask API routing and SQLAlchemy queries, and help understand the client-server architecture.

Video:

https://drive.google.com/file/d/1Fg7ktNcN3_u698P9d2g4mv010q4pwiGT/view?usp=sharing